SAGE UNIVERSITY BHOPAL
SCHOOL OF COMPUTER TECHNOLOGY

COMPREHENSIVE PAGE REPLACEMENT ALGORITHM SIMULATOR & PERFORMANCE ANALYSIS FRAMEWORK

Operating Systems Laboratory Project Report

Student Name: Anuj Prajapati

Enrollment Number: BTE25DTA000008

Course Name: Operating Systems Lab

Academic Session: 2026-27

1. INTRODUCTION AND THEORETICAL BACKGROUND

1.1 Introduction to Virtual Memory Management

Modern operating systems utilize virtual memory management to allow execution of processes that may not be completely loaded in physical primary memory (RAM). This architectural breakthrough decouples the logical address space perceived by an application from the physical memory addresses available on hardware. When a process attempts to reference a memory page that is currently not resident in a physical frame, a page fault occurs. The operating system's virtual memory manager must then interrupt the CPU, invoke the page fault handler, locate the requested page on secondary backing storage (such as an SSD or HDD), and load it into an available physical frame.

If all physical frames are currently allocated, the operating system must evict an existing resident page to accommodate the incoming page. The mechanism governing which page is chosen for eviction is known as a page replacement algorithm. The design and efficiency of these algorithms are critical; improper eviction policies lead to excessive disk I/O swapping—a degenerative condition known as thrashing—which severely degrades system throughput and application responsiveness.

1.2 Objectives of the Simulator Project

The primary objective of this project is to engineer an interactive, robust, and extensible simulation engine in Python that models, evaluates, and contrasts the execution behavior of three classic page replacement algorithms:

First-In, First-Out (FIFO): Evicts the oldest resident page based on load timestamp.

Least Recently Used (LRU): Evicts the page that has not been referenced for the longest duration in past execution history.

Optimal Page Replacement (OPT / MIN): Evicts the page that will not be referenced for the longest period in future execution (serving as an unattainable theoretical benchmark).

Furthermore, the project aims to verify Belady's Anomaly—a counterintuitive phenomenon where increasing the number of allocated physical frames paradoxically results in an increase in the total number of page faults for specific reference strings under FIFO execution.

2. SYSTEM ARCHITECTURE & TECHNICAL SPECIFICATIONS

2.1 Core Architectural Modules

The simulator is structured as a modular object-oriented and functional Python codebase divided into core execution pipelines:

Module Component	File Association	Core Responsibility & Functionality
Core Simulator Logic	page_replacement.py	Implements algorithmic state machine, frame matrix tracking, hit/fault counters, and comparative metric synthesis.
Interactive Notebook	Page_Replacement_Simulator.ipynb	Jupyter environment housing experimental test suites, manual traces, and visual output tables.
Assignment Documentation	Assignment_Answers.md	Comprehensive answers to theoretical operating system questions, manual trace breakdowns, and anomaly proofs.
Execution Logs	Sample_Execution_Output.txt	Raw terminal output logs capturing trace matrices across diverse reference strings and frame capacities.

2.2 Input Validation & Parsing Subsystem

To ensure robust interactive execution, the simulator implements rigorous input sanitization. Users can provide reference strings separated by spaces or commas. The input parser automatically strips extraneous whitespace, casts string tokens to integer page numbers, validates that frame allocations are positive integers, and flags malformed inputs with descriptive error exceptions.

3. DETAILED ALGORITHMIC IMPLEMENTATION

3.1 First-In, First-Out (FIFO) Algorithm

FIFO maintains a queue of all pages currently in memory. When a page must be replaced, the page at the head of the queue (the one that entered earliest) is selected for eviction. The simulator maintains a Python list representing frames in chronological order of arrival. Upon encountering a page fault, if frames are not at full capacity, the new page is appended. If full, the page at index 0 is popped and the new page is appended to the rear.

3.2 Least Recently Used (LRU) Algorithm

LRU approximates optimal performance by exploiting temporal locality of reference. Upon a page hit, the referenced page is accessed, removed from its current position, and pushed to the top (most recently used). Upon a page fault with full frames, the page at the bottom of the stack (least recently used) is evicted.

3.3 Optimal (OPT) Page Replacement Algorithm

Formulated by Laszlo Belady, the Optimal algorithm looks forward in the reference string and evicts the page whose next reference occurs furthest in the future. It guarantees the absolute minimum number of page faults for any given reference string and frame count.

4. COMPREHENSIVE EXPERIMENTAL SIMULATION & TRACE ANALYSIS

4.1 Standard Test Reference String Execution

To evaluate comparative performance, the simulator was executed using standard reference string 1 2 3 1 4 2 5 with a physical frame capacity of 3.

4.2 Step-by-Step FIFO Trace Matrix

Step	Reference	Frame 1	Frame 2	Frame 3	Status	Evicted
1	1	1	-	-	Fault	None
2	2	1	2	-	Fault	None
3	3	1	2	3	Fault	None
4	1	1	2	3	Hit	None
5	4	4	2	3	Fault	1
6	2	4	2	3	Hit	None
7	5	4	5	3	Fault	2

FIFO Summary Metrics: Total Page Faults = 5 | Total Page Hits = 2 | Hit Ratio = 28.57% | Fault Ratio = 71.43%

4.3 Step-by-Step LRU Trace Matrix

Step	Reference	Frame 1	Frame 2	Frame 3	Status	Evicted
1	1	1	-	-	Fault	None
2	2	1	2	-	Fault	None
3	3	1	2	3	Fault	None
4	1	1	2	3	Hit	None
5	4	1	4	3	Fault	2
6	2	1	4	2	Fault	3
7	5	5	4	2	Fault	1

LRU Summary Metrics: Total Page Faults = 6 | Total Page Hits = 1 | Hit Ratio = 14.29% | Fault Ratio = 85.71%

4.4 Step-by-Step Optimal (OPT) Trace Matrix

Step	Reference	Frame 1	Frame 2	Frame 3	Status	Evicted
1	1	1	-	-	Fault	None
2	2	1	2	-	Fault	None
3	3	1	2	3	Fault	None
4	1	1	2	3	Hit	None
5	4	1	2	4	Fault	3
6	2	1	2	4	Hit	None
7	5	5	2	4	Fault	1

Optimal Summary Metrics: Total Page Faults = 5 | Total Page Hits = 2 | Hit Ratio = 28.57% | Fault Ratio = 71.43%

5. COMPARATIVE PERFORMANCE ANALYSIS & BENCHMARKING

5.1 Comparative Metric Synthesis

Algorithm	Total References	Page Faults	Page Hits	Hit Ratio (%)	Fault Ratio (%)
FIFO	7	5	2	28.57%	71.43%
LRU	7	6	1	14.29%	85.71%
Optimal (OPT)	7	5	2	28.57%	71.43%

5.2 Key Takeaways and Architectural Trade-offs

FIFO Simplicity vs. Anomalies: FIFO is computationally trivial to implement using a simple pointer or queue structure, but its blind adherence to arrival time causes suboptimal evictions and susceptibility to Belady's Anomaly.

LRU Adaptability: LRU closely approximates optimal performance in programs exhibiting strong temporal locality. However, true hardware implementation requires expensive timestamp registers or stack update linked lists.

Optimal Benchmark: While unimplementable in real-world operating systems due to the impossibility of predicting future execution paths, OPT serves as an absolute ceiling for evaluating practical heuristic algorithms.

6. INVESTIGATION OF BELADY'S ANOMALY

6.1 Mathematical Proof & Experimental Verification

Belady's Anomaly is a famous counterintuitive finding in computer architecture demonstrating that for certain memory reference strings, increasing the number of allocated physical frames can cause an increase in page faults under FIFO replacement. Consider the reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5.

Execution with 3 Frames (FIFO): Generates 9 page faults.

Execution with 4 Frames (FIFO): Generates 10 page faults.

7. SOURCE CODE IMPLEMENTATION

The core Python script powering the simulator engine is structured as follows:

```
# Page Replacement Algorithm Simulator
# Author: Anuj Prajapati (Roll No: BTE25DTA000008)

def simulate_fifo(pages, capacity):
    frames = []
    faults = 0
    hits = 0
    for page in pages:
        if page not in frames:
            if len(frames) < capacity:
                frames.append(page)
            else:
                frames.pop(0)
                frames.append(page)
            faults += 1
        else:
            hits += 1
    return faults, hits

def simulate_lru(pages, capacity):
    frames = []
    faults = 0
    hits = 0
    for page in pages:
        if page in frames:
            frames.remove(page)
            frames.append(page)
            hits += 1
        else:
            if len(frames) < capacity:
                frames.append(page)
            else:
                frames.pop(0)
                frames.append(page)
            faults += 1
    return faults, hits

def simulate_optimal(pages, capacity):
    frames = []
```

```
faults = 0
hits = 0
for i, page in enumerate(pages):
    if page in frames:
        hits += 1
    else:
        if len(frames) < capacity:
            frames.append(page)
        else:
            farthest = -1
            index_to_replace = -1
            for j, frame in enumerate(frames):
                if frame not in pages[i+1:]:
                    index_to_replace = j
                    break
                else:
                    next_use = pages[i+1:].index(frame)
                    if next_use > farthest:
                        farthest = next_use
                        index_to_replace = j
            frames[index_to_replace] = page
        faults += 1
return faults, hits
```

8. CONCLUSION AND FUTURE ENHANCEMENTS

8.1 Conclusion

This project successfully designed, implemented, and evaluated a comprehensive Page Replacement Algorithm Simulator. By examining FIFO, LRU, and Optimal algorithms through rigorous trace matrices and comparative metrics, the project highlights the fundamental tradeoffs between algorithmic simplicity and memory management efficiency.

8.2 Future Scope

GUI Integration: Developing a responsive desktop or web GUI for real-time visualization of frame swapping.

Advanced Algorithms: Extending the simulator to support Second-Chance (Clock), Not Frequently Used (NFU), and Working Set algorithms.

9. REFERENCES

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). John Wiley & Sons.

Tanenbaum, A. S., & Bos, H. (2015). Modern Operating Systems (4th ed.). Pearson.

Belady, L. A. (1966). A study of replacement algorithms for a virtual-storage computer. IBM Systems Journal, 5(2), 78-101.

SAGE University Bhopal (2026). SAGE University Bhopal - Operating Systems Laboratory Manual (2026).